Embedded Data Structure
(Lecture Note 4)

Revisit C Programming (3)

2025. 3. 13

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Basic concept of advanced operations for pointers**
  - **Relationship between array and pointer**
  - **Pointer and function / void pointer (casting)**
  - **Function pointer / pointer of pointer**

    ※ Remember the definition of "pointer of pointer"

  - **Interpret the following statement :**

```
int p;  /  int *p;  /  int **p;
```

Pointer and String (1/5)

KU KONKUK UNIVERSITY

▪ What is "string" ?

- **String vs. Character (What is the main difference ?)**

  - String : a set of character ? (represented by double quotes " ")

    cf) Character is represented by single quotes ' '

Examples of string :

"a", " ", "#$%!", "안녕하세요" // blank can be string

"Welcome to C programming !"

- **How can we store string in C ?**

  - We can use **char** (array)

    ※ Size of array needs to be set as LENGTH + 1 due to **NULL**

H	E	L	L	0	₩0

→ **Why do we need it ?**

Pointer and String (2/5)

KU KONKUK UNIVERSITY

▪ What is "string" ? – cont'd

- **There are various approaches for initializing strings**

`char str[4] = {'a', 'b', 'c', '₩0'};`	str	a	b	c	₩0	
`char str[4] = "abc";`	str	a	b	c	₩0	
`char str[6] = "abc";`	str	a	b	c	₩0	₩0	₩0
`char str[4] = "";`	str	₩0	₩0	₩0	₩0	
`char str[] = "abc";`	str	a	b	c	₩0	

- String constant (e.g., "abc") → NULL is automatically set by compiler
- [] assigns automatically the size of the corresponding array

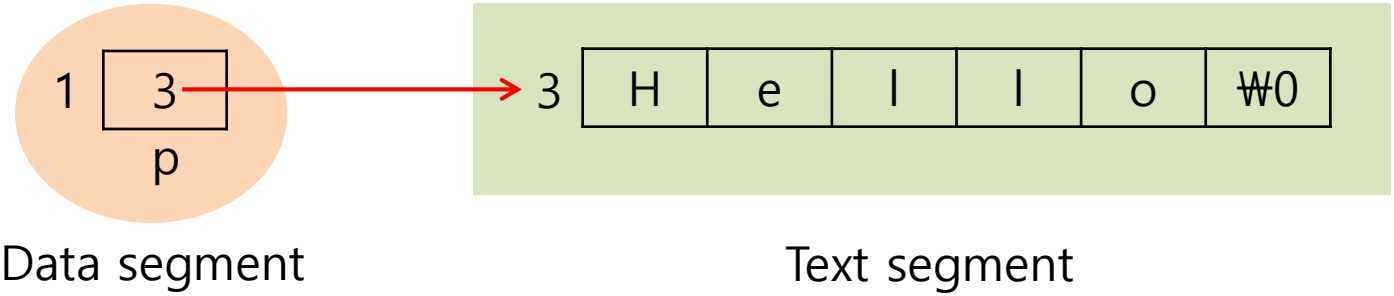
▪ String store by pointer

- **String constant : string included in your source (e.g., "abc")**

※ String constant is stored in the **text segment**

→ **read only ! (cannot modified)**

```
char *p = "Hello";
```



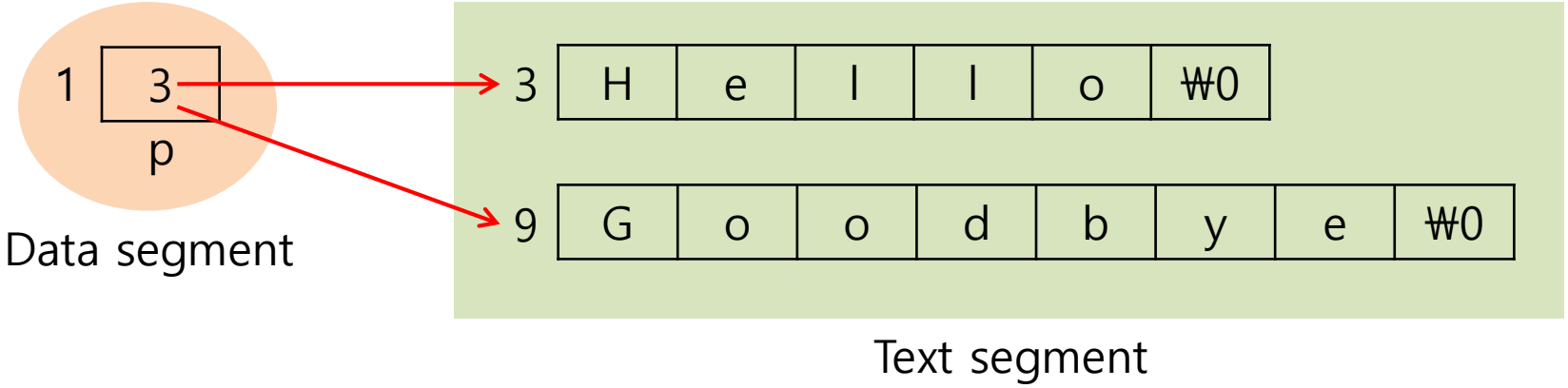

```
char *p = "Hello";
strcpy(p, "Goodbye"); // string copy
```

➡ **Is it Ok ? (see the above)**

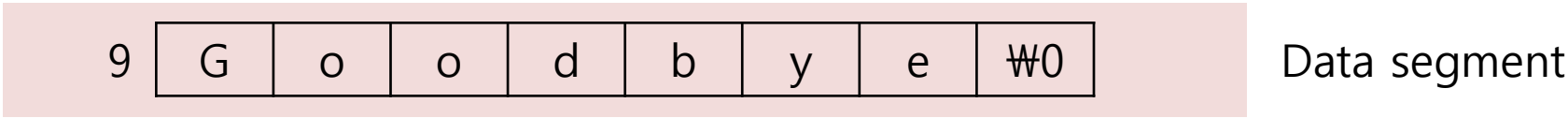
▪ String store by pointer – cont'd

- **String constant : a kind of address**

```
char *p = "Hello";
p = "Goodbye"; // copy address ! (not the context)
```




```
char p[] = "Hello";
strcpy(p, "Goodbye");
```



Pointer and String (5/5)

- **Example : string print-out**
  - **Consider the following example with the call-by-reference**

```
#include <stdio.h>

void set_pointer(char *q);
char *st = "Good !";

void main()
{
    char *p = "aaa";
    set_pointer(p);
    printf("%s₩n", p);
}

void set_pointer(char *q)
{
    q = st;
}
```

Your selection ⟷

```
#include <stdio.h>

void set_pointer(char **q);
char *st = "Good !";

void main()
{
    char *p = "aaa";
    set_pointer(&p);
    printf("%s₩n", p);
}

void set_pointer(char **q)
{
    *q = st;
}
```

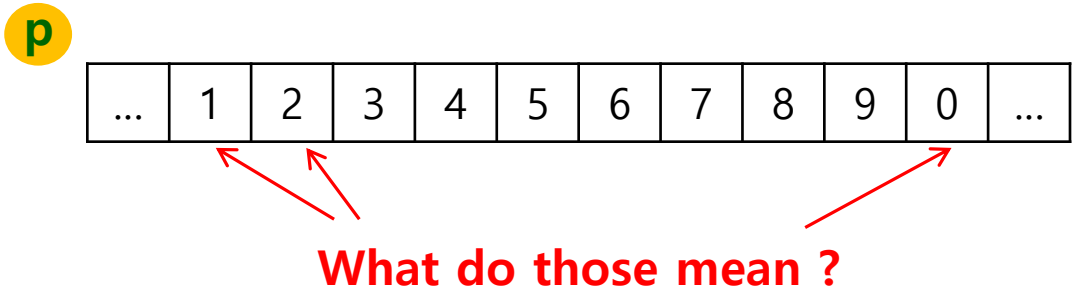
Array of Pointers (1/3)

- **Array containing pointers as elements**
  - **Interpret the following declaration :**

Contain int-type pointers

```
int *p[10];
```

p is an array ([ ] > *)




```
// Initialize array

int a, b, c, d;
int *p[5] =
      {&a, &b, &c, &d};
```

Revisit: Function Pointer (1/2)

- **Function includes the address**
  - **Note : function pointer indicates function and its parameters**

Return type (integer)

```
int (*fptr)(int);
```

Parameters

fptr is a pointer

```
#include <stdio.h>

void sum(float, float);
```

(Function pointer)

```
void sum(float a, float b)
{
  float result;
  result = a + b;
}
```

* Name of function pointer
: not limited

Revisit: Function Pointer (2/2)

KU KONKUK UNIVERSITY

▪ Implementation example

- **Implement the following code (not function call, but function pointer)**

```c
#include <stdio.h>

int add(int x, int y)
{
    int result;
    result = x + y;
    return result;
}

int sub(int x, int y)
{
    int result;
    result = x – y;
    return result;
}
```

```c
void main()
{
    int result;
    [                    ]  ← Declaration

    [                    ]  ← For add
    result = fptr(10, 20);
    printf("%d₩n", result);

    [                    ]  ← For sub
    result = fptr(10, 20);
    printf("%d₩n", result);
}
```

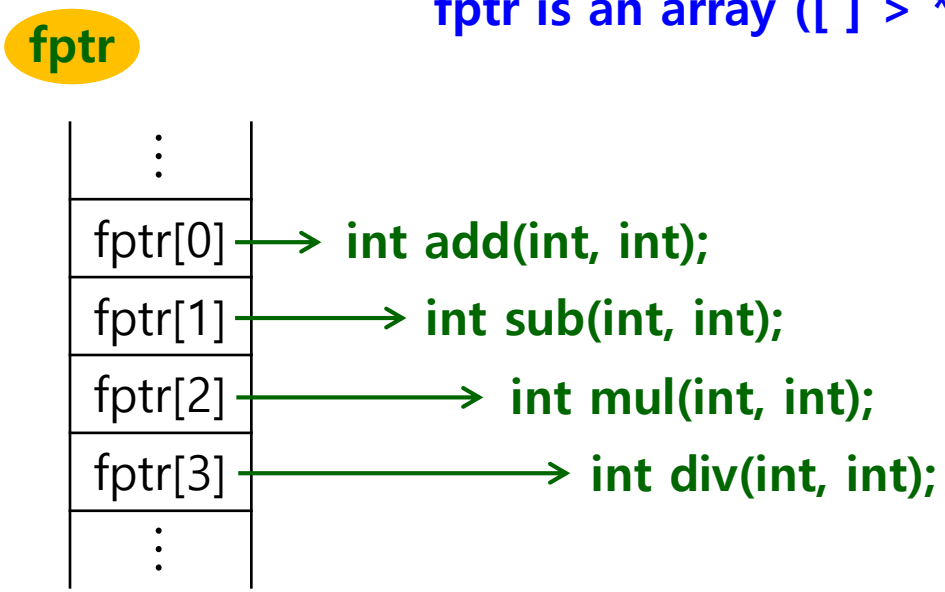
➡ **Make complete your code**

Array of Function Pointers

- **Definition of array of function pointers**
  - **Add [] to indicate the array !**

```
int (*fptr[4])(int);
```

fptr is an array ([ ] > *) for what ?



※ Why do we use this ?

➔ Routine for "menu"

※ Compare it with array of pointers

File Open/Close (1/3)

- **fopen()/fclose() functions**
  - **We can read a specific file into our source code**

```
FILE *fopen(const char *name, const char *mode)
```

```
int fclose(FILE *stream)
```

file name

mode

```
FILE *fp;
fp = fopen("test.txt", "w");
fclose(fp);
```

If file does not exist,

fopen() returns NULL

[Example of modes]

Mode	Description
"r"	Read file
"w"	Write (generate) file
"rt" / "rb"	Read file (text/binary)
"wt"/ "wb"	Write file (text/binary)

File Open/Close (2/3)

- **Implementation examples**
  - **Check the following code (file read)**

```
#include <stdio.h>

void main()
{
    FILE *fp;
    fp = fopen("test.txt", "r");

    if(fp == NULL)
        printf("There is no file₩n");
    else
        printf("Success !₩n");
}
```

※ remove(filename)

- Return 0 when removing a file
- Return -1 otherwise

[Useful functions]

Mode	Description
int foef(FILE *strem)	Return true at the end
int rename(const char *o, const char *n)	Change the file name
FILE *tmpefile()	Temporary file gen.
int ferror(FILE *strem(	If error occurs, return true

File Open/Close (3/3)

- **Formatted read/write**
  - **What is difference btw fprintf() and printf() ?**

```
int fprintf(FILE *fp, const char *format, ...);
```

```
int fscanf(FILE *fp, const char *format, ...);
```

```
void main()
{
    FILE *fp;

    fp = fopen("score.txt", "w");
    while(1){
        if(score < 0) break;
        printf("Input score: ");
        scanf("%d", &score);
        fprintf(fp, "%d₩n", score);
    }
    fclose(fp);
```

```
if((fp = fopen("score.txt", "r") != NULL)
{
    while(!feof(fp)){
        fscanf(fp, "%d", &score);
        avg += score;
        num++;
    }
    avg /= (float)num;
    printf("average = %f₩n", avg);
}
```

➡ **Complete code for average !**

Summary

- **Basic concept of advanced operations for pointers**
  - **Pointer and string**
    - Inputs of functions related to string are pointers in general
    - Remember the relation between pointer of pointer and string
  - **Pointer and its applications**
    - Array of pointers, function pointer, and array of function pointers
  - **Remember how to read/write a file**